

CompSci 234/NetSys 210
Advanced Topics in Networking
Winter 2019

Unit 2: Application Layer Protocols

1

Cheng-Hsin Hsu (chsu@cs.nthu.edu.tw)

Slide adopted from Kurose/Ross

Agenda

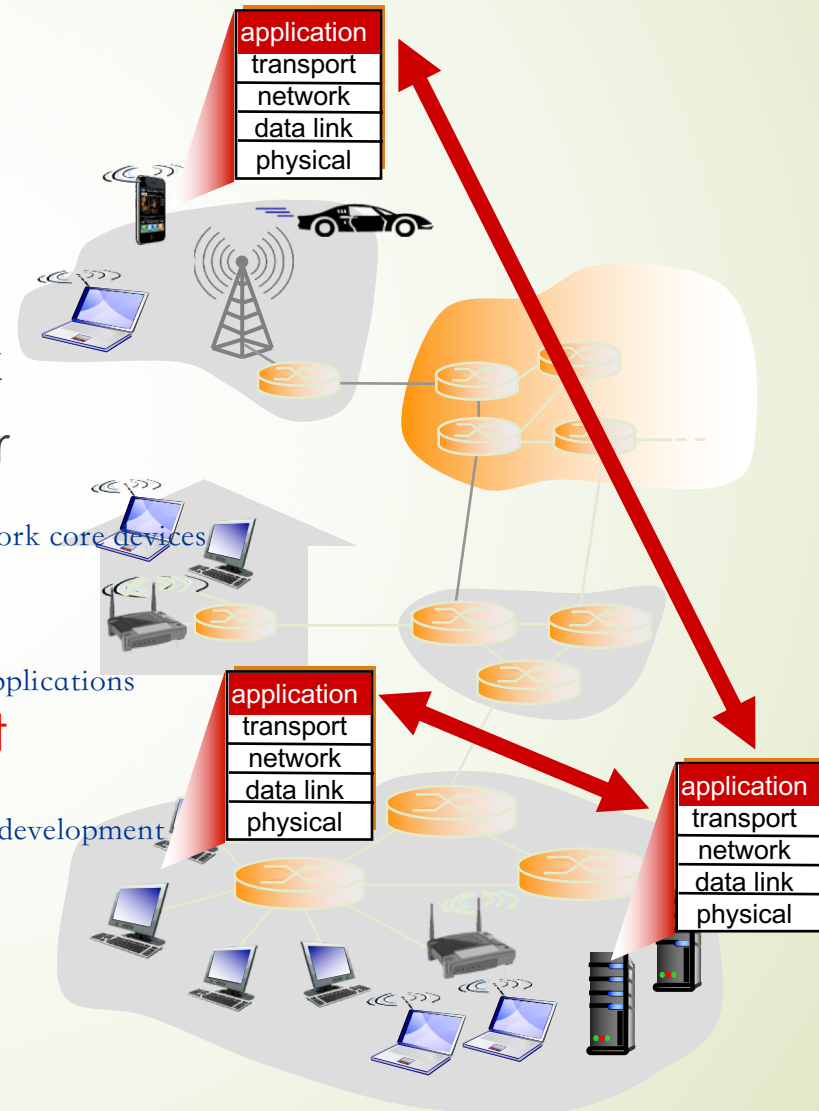
- Architecture: Client-Server versus P2P
- Application Service Requirements: TCP versus UDP
- Sample Application Protocol: HTTP
- Socket Programming in Python

Sample Network Applications

- e-mail
- web
- text messaging
- remote login
- P2P file sharing
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- voice over IP (e.g., Skype)
- real-time video conferencing
- social networking
- search
- Internet-of-things (IoT)
- pub/sub
- ...

How to Write a Network Application

- a network program:
 - runs on (different) end systems
Runs on different end systems
 - communicates over network
Communicates over network
 - e.g., web server and browser software
No need to write software for network core devices
- no need to write software for network-core devices
Network-core devices do not run user applications
 - **network-core devices do not run user applications**
Applications on end systems allows for rapid app development
 - applications on end systems allows for rapid app development, propagation
Propagation



Application Architecture

- Architecture is not the implementation itself, but “**design blueprint**” on how to “**organize**” instead

- what interfaces are supported
- where each functionality is implemented

- Application architectures

- Client-to-Server Client to server
- Peer-to-Peer (P2P) Peer to peer

What interfaces are supported

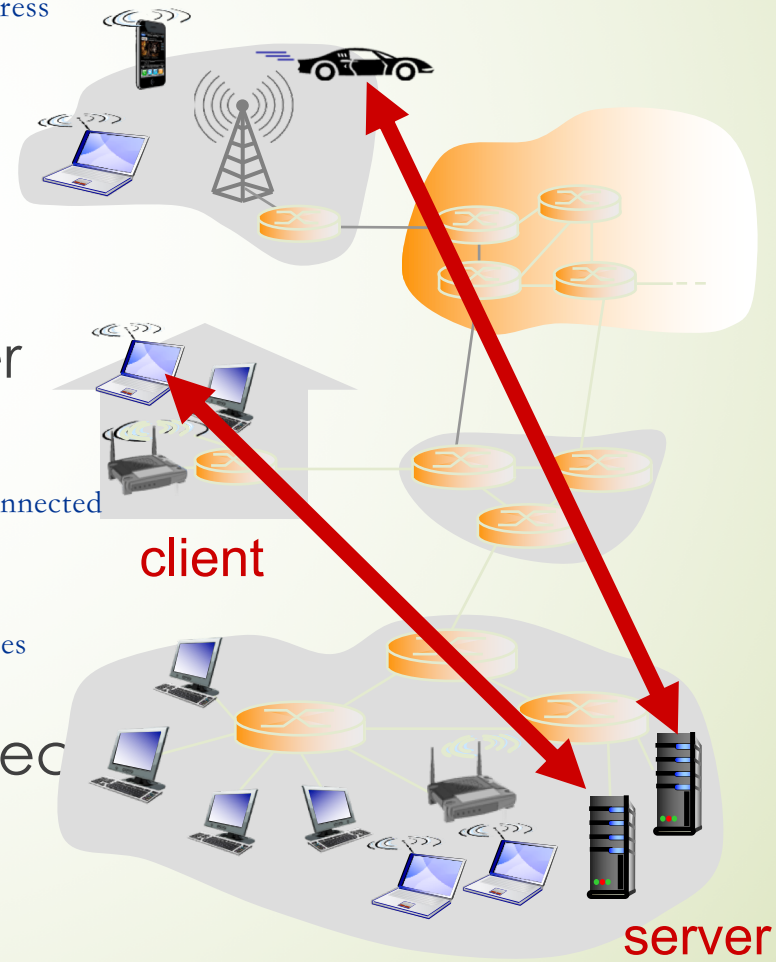
Where each functionality is implemented

General...
more than
app. arch.



Client-Server Architecture

- server:
 - Always on host
 - always-on host Permanent IP address
 - permanent IP address
 - data centers for scaling Data centers for scaling
- clients:
 - Communicate with server
 - communicate with server
 - may be intermittently connected May be intermittently connected
 - may have dynamic IP addresses May have dynamic ip addresses
 - do not communicate directly with each other Do not communicate directly with each other



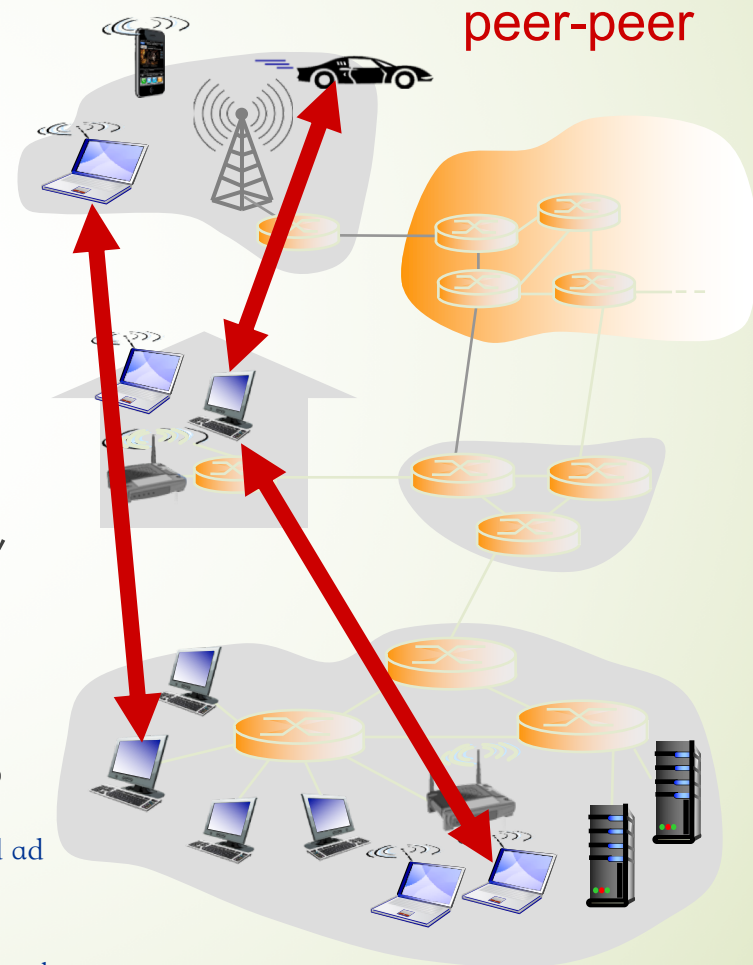
P2P Architecture

no always on server

- no always-on server
Arbitrary end systems directly communicate
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
Peer request service from other peers
provide service in return to other peers
- self scalability – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
Peers are intermittently connected and change ip addresses
- complex management

Self scalability
New peers bring new service capacity
As well as new service demands

Complex management



Agenda

- Architecture: Client-Server versus P2P
- Application Service Requirements: TCP versus UDP
- Sample Application Protocol: HTTP
- Socket Programming in Python

Application Protocol Defines...

Types of messages | message syntax | message semantics | rules for processes | open protocols

Rfcs allow for interoperability proprietary protocols

- **types** of messages exchanged,
 - e.g., request, response
- message **syntax**:
 - what fields in messages & how fields are delineated
- message **semantics**
 - meaning of information in fields
- **rules** for when and how processes send & respond to messages
- **open** protocols:
 - defined in RFCs
 - allows for interoperability
 - e.g., HTTP, SMTP
- **proprietary** protocols:
 - e.g., Skype

Transport Service Requirements

Data integrity

timing

throughput

Security

data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

timing

- ➡ some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- other apps (“elastic apps”) make use of whatever throughput they get

security

- encryption, data integrity, ...

Typical Applications and Their Requirements

application	data loss	throughput	time critical
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video: 10kbps-5Mbps	yes, 100' s msec
stored audio/video	loss-tolerant	same as above	yes, few secs
networked games	loss-tolerant	few kbps up	yes, 100' s msec
text messaging	no loss	elastic	yes and no

Q: Why networked gaming traffic is loss tolerant?

TCP versus UDP

➤ TCP service:

Reliable transport between
sending and receiving process

Flow control sender won't
overwhelm receiver

Congestion control
Throttle sender when network
Overloaded

Connection-oriented
Setup required between client
and server processes

Does not provide timing
minimum throughput
guarantee security

- **reliable** transport between sending and receiving process
- **flow control**: sender won't overwhelm receiver
- **congestion control**: throttle sender when network overloaded
- **connection-oriented**: setup required between client and server processes
- **does not provide**: timing, minimum throughput guarantee, security

➤ UDP service:

- **unreliable** data transfer between sending and receiving process

- **does not provide**: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup,

- **Q: why bother? Why is there a UDP?**

Transport Protocols Used by Typical Applications

application	application layer protocol	underlying transport protocol
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	HTTP (e.g., YouTube), RTP [RFC 1889]	TCP or UDP
Internet telephony	SIP, RTP, proprietary (e.g., Skype)	TCP or UDP

Q: Multimedia streaming and Internet telephony used to be on UDP; but now on both. Why?

Secured TCP

- TCP & UDP
 - no encryption
 - cleartext passwords sent into socket traverse Internet in cleartext
- SSL: Secured Socket Layer
 - provides encrypted TCP connection
 - data integrity
 - end-point authentication

Secured socket layer

Provides encrypted
tcp connection

Data integrity

End-point
authentication

Ssl is at app layer

- SSL is at app layer
 - apps use SSL libraries, that “talk” to TCP
- SSL socket API
 - cleartext passwords sent into socket traverse Internet encrypted

Ssl socket api: cleartext passwords sent
into socket traverse internet encrypted

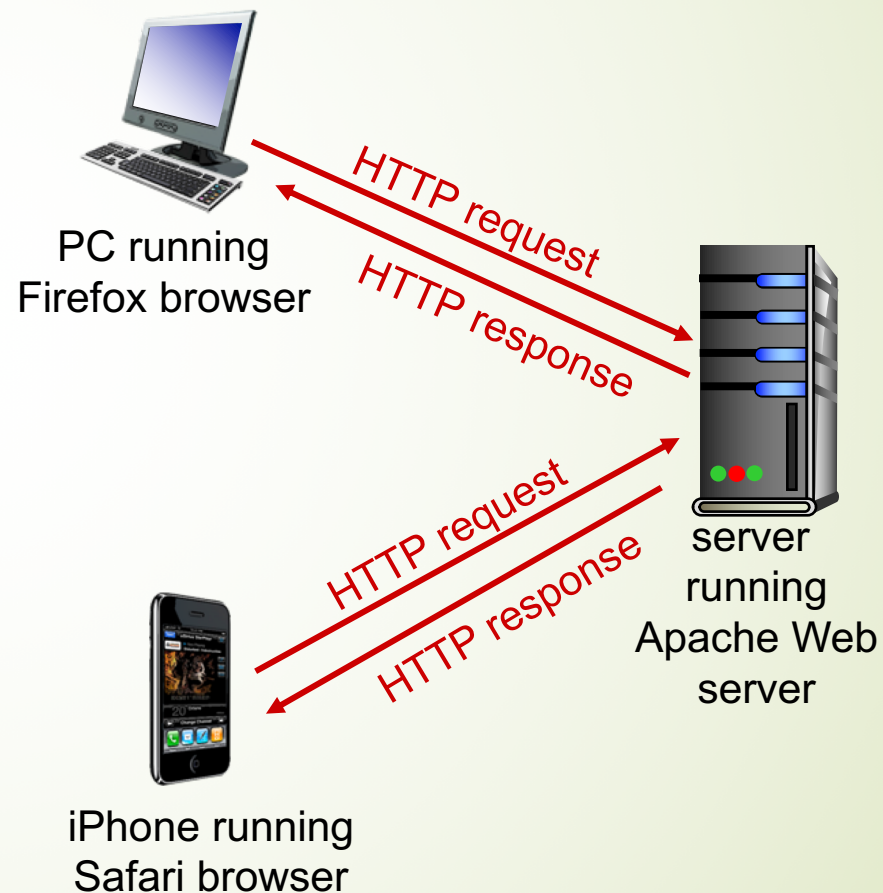
Agenda

- Architecture: Client-Server versus P2P
- Application Service Requirements: TCP versus UDP
- Sample Application Protocol: HTTP
- Socket Programming in Python

What is HTTP?

Hypertext transfer protocol

- HTTP: hypertext transfer protocol
- Web's application layer protocol
- Web's application layer protocol
- client/server model
 - **client**: browser that requests, receives, (using HTTP protocol) and "displays" Web objects
 - **server**: Web server sends (using HTTP protocol) objects in response to requests



Properties of HTTP

tcp

- Use TCP, server port 80
- HTTP clients and servers exchange **messages**

Messages

- HTTP is **stateless**
- protocols that maintain “state” are complex!

Stateless

- past history (state) must be maintained
- if server/client crashes, their views of “state” may be inconsistent, must be reconciled
- scalability concern

Persistent and Non-Persistent HTTP

➤ non-persistent HTTP

- at most one object sent over TCP connection
- connection then closed
- downloading multiple objects required multiple connections

At most one object sent over tcp connection

Connection then closed

Downloading multiple objects
Required multiple connections

➤ persistent HTTP

- multiple objects can be sent over single TCP connection between client, server

Multiple objects can be sent over single tcp connection between client and server